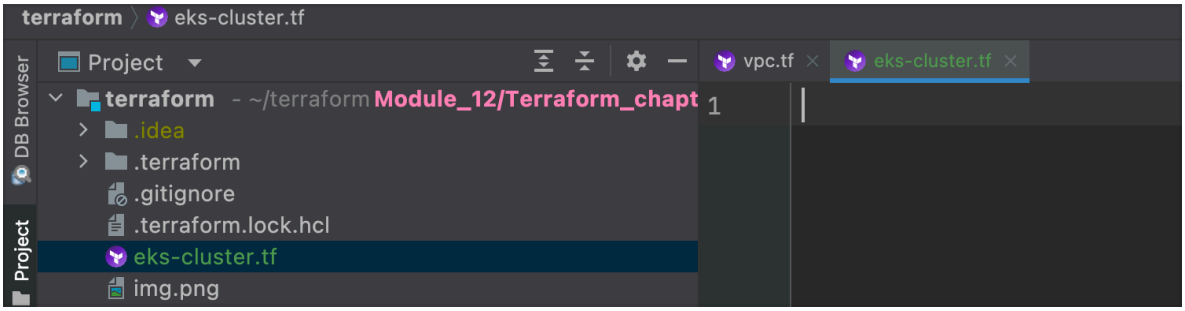
Module 12 - chapter 20

Automate Provisioning EKS cluster with Terraform - Part 2

EKS cluster and Worker Nodes

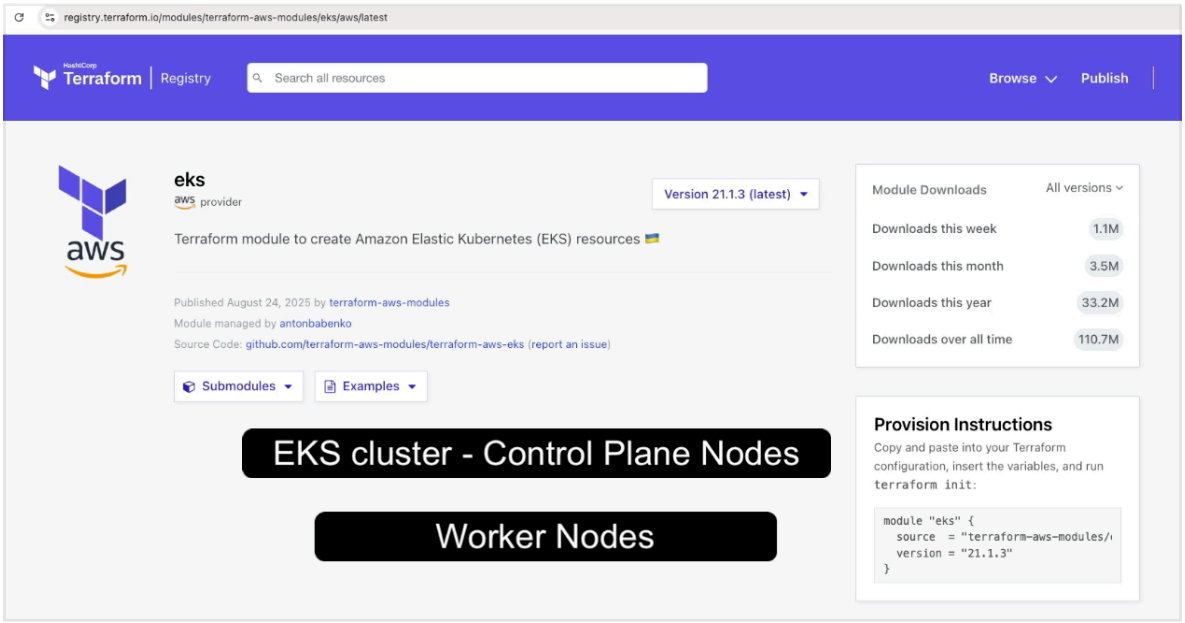
With the VPC configured we can now create the EKS cluster.

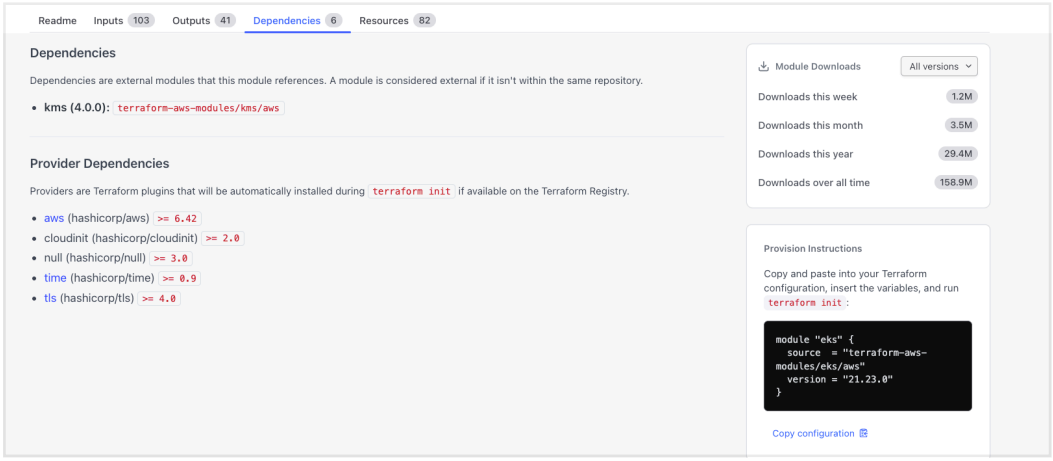
So I create a new .tf file called ‘eks-cluster.tf’



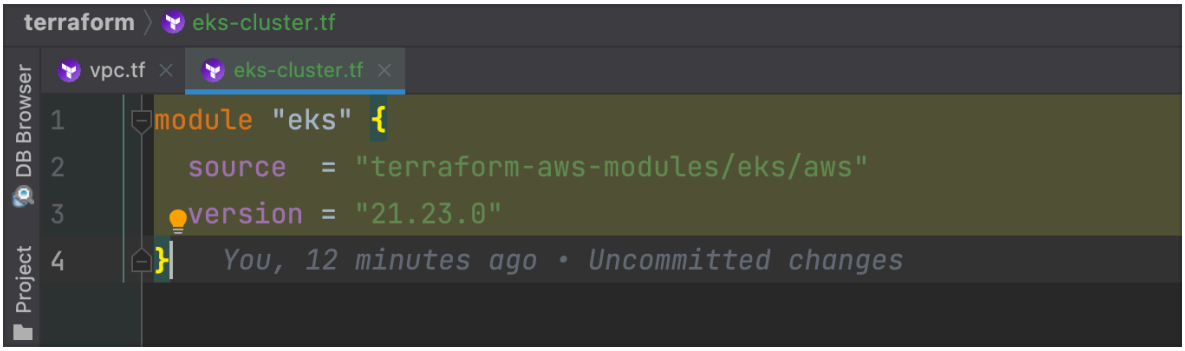
And we will use an AWS eks tf module to provision

- EKS cluster
- Worker nodes

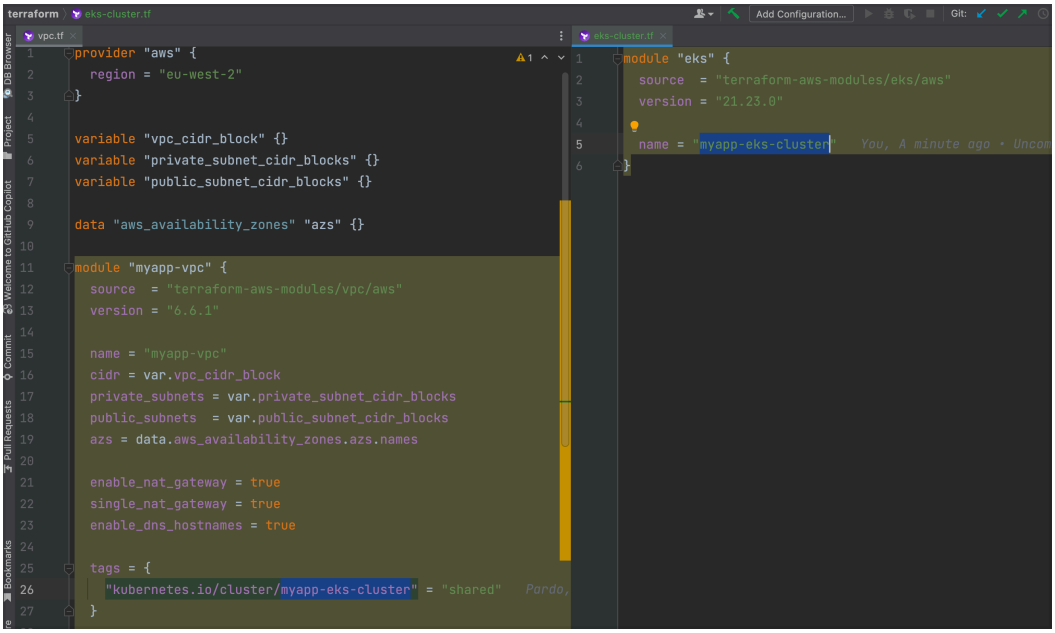




And we copy the basic syntax into my tf code:

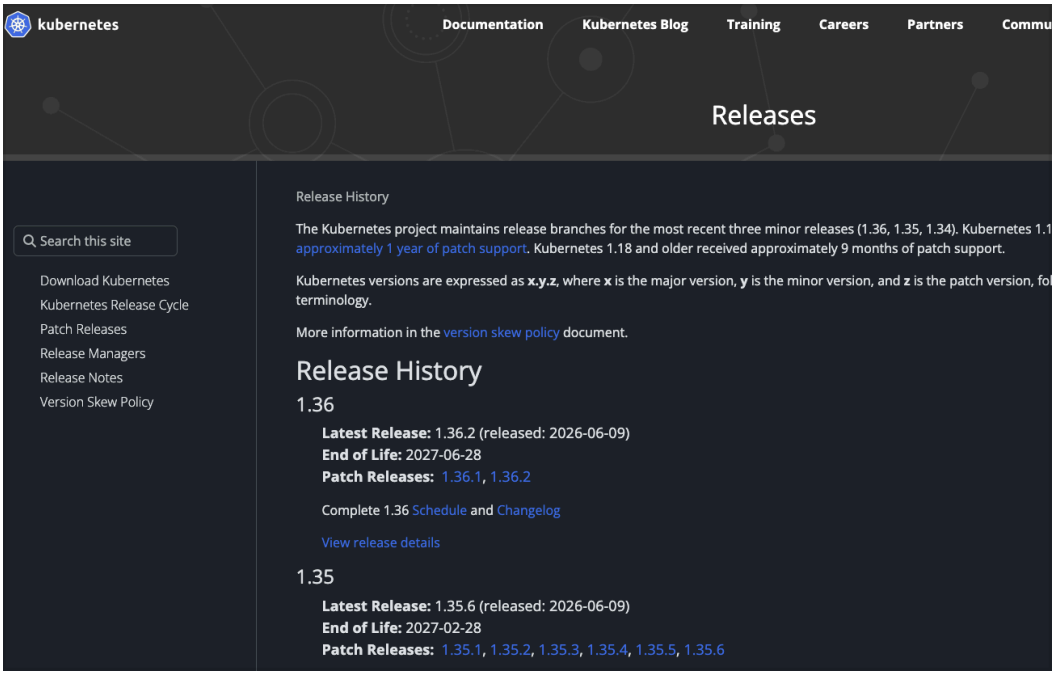


Now we set the name of the cluster, which we already define in vpc.tf

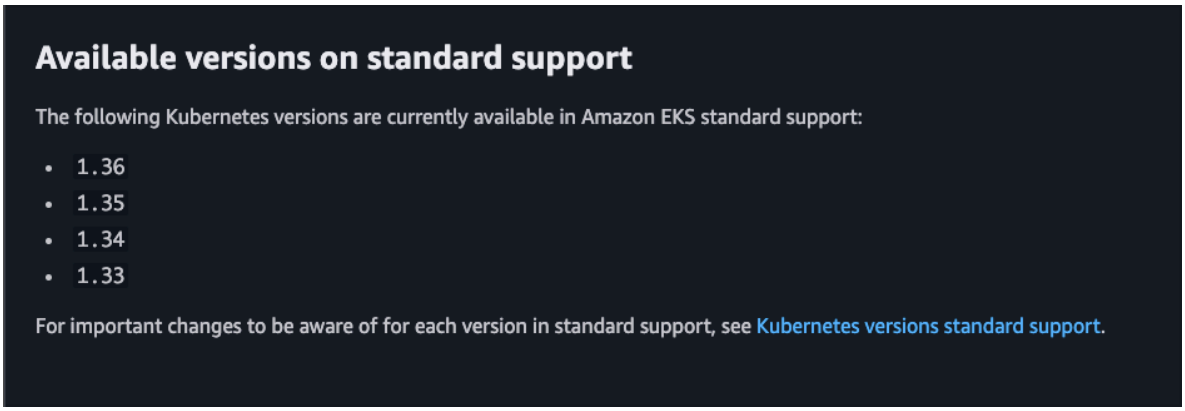


Then we set the Kubernetes version:

https://kubernetes.io/releases/



https://docs.aws.amazon.com/eks/latest/userguide/kubernetes-versions.html



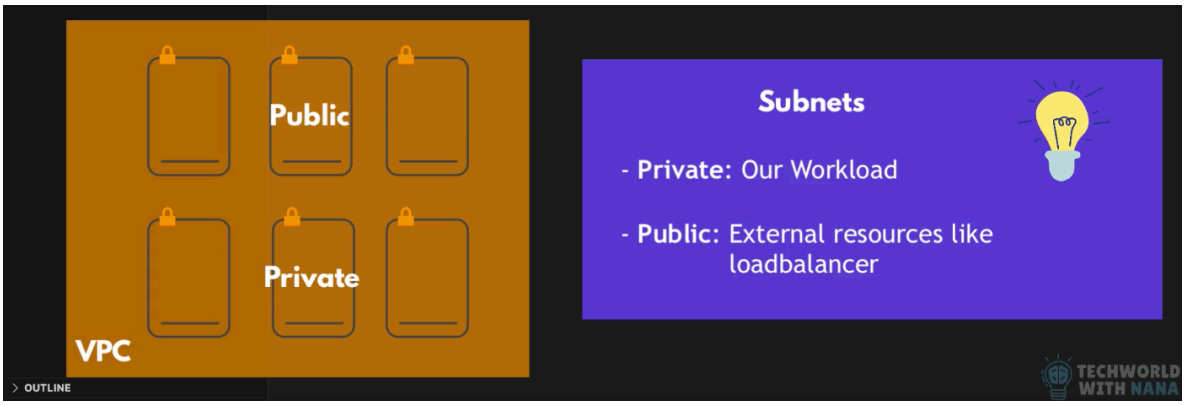
So I use 1.36

Now we set ‘subnet_ids’ -> list of subnets where we want the workers nodes to be started in.

So currently we have 6 subnets, 3 public and 3 public. Each AZ has 1 private and one public.

And we want our workload to be scheduled in the private subnets.

The public subnets are for external resources like loadbalancer



We check the documentation and in the vpc module documentation when we check the output section can see that the 'private_subnets attribute is an output so we can call it!

->subnet_ids = module.myapp-vpc.private_subnets

```
eks-cluster.tf
1  module "eks" {
2    source  = "terraform-aws-modules/eks/aws"
3    version = "21.23.0"
4
5    name = "myapp-eks-cluster"
6    kubernetes_version = "1.36"
7
8    subnet_ids = module.myapp-vpc.private_subnets
9  }   You, Moments ago • Uncommitted changes
```

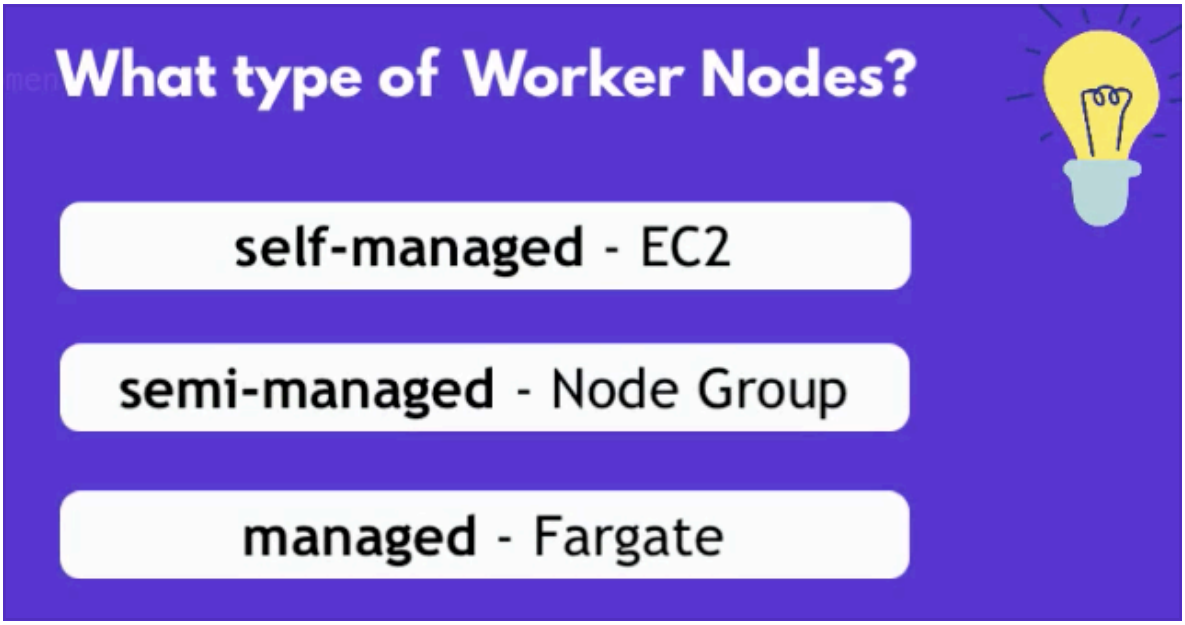
For the eks cluster module we don't have to set tags but we will

```
9
10      tags = {
11        "Environment" = "dev"
12        application   = "myapp"    You, 2
13      }
14  }
```

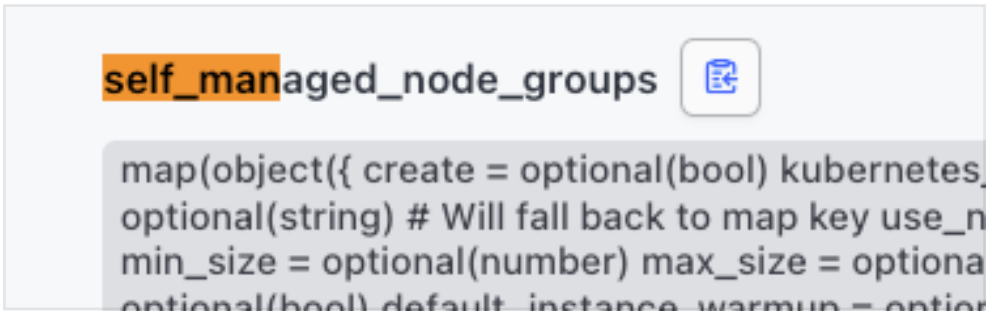
We set the vpc_id using a reference to the vpc module

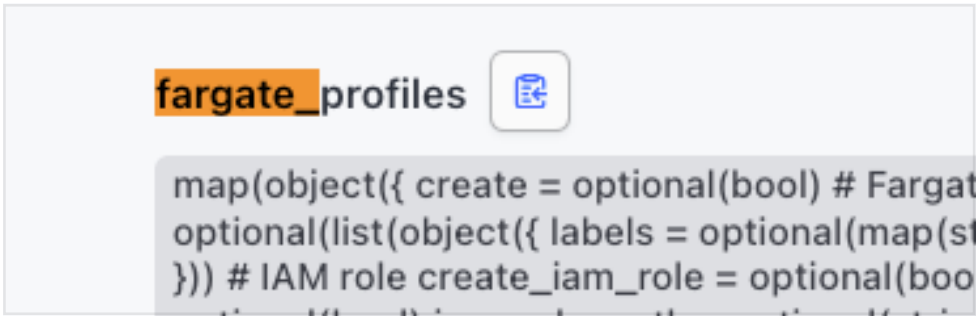
```
7
8       subnet_ids = module.myapp-vpc.private_subnets
9       vpc_id     = module.myapp-vpc.vpc_id    You, Moments
```

We new to configure the worker nodes -> what type of worker node we want to run?

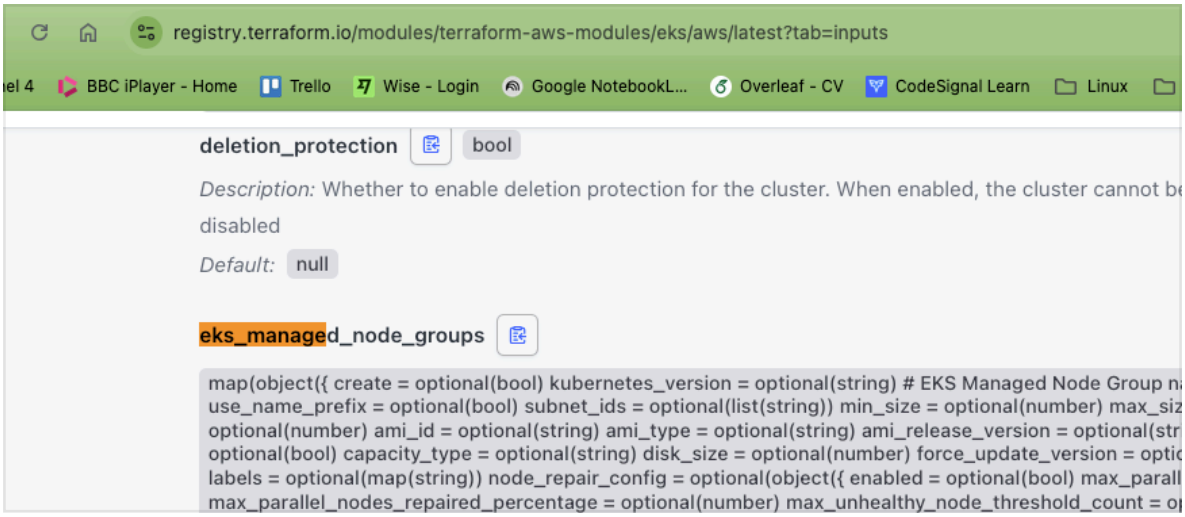


We can find the sel_managed and the fargate in the eks tf module:

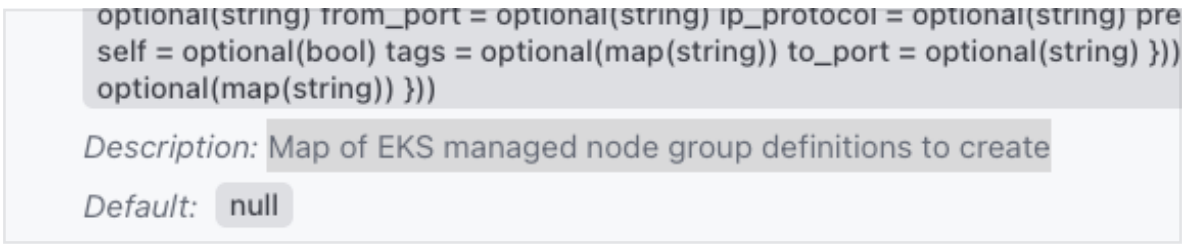




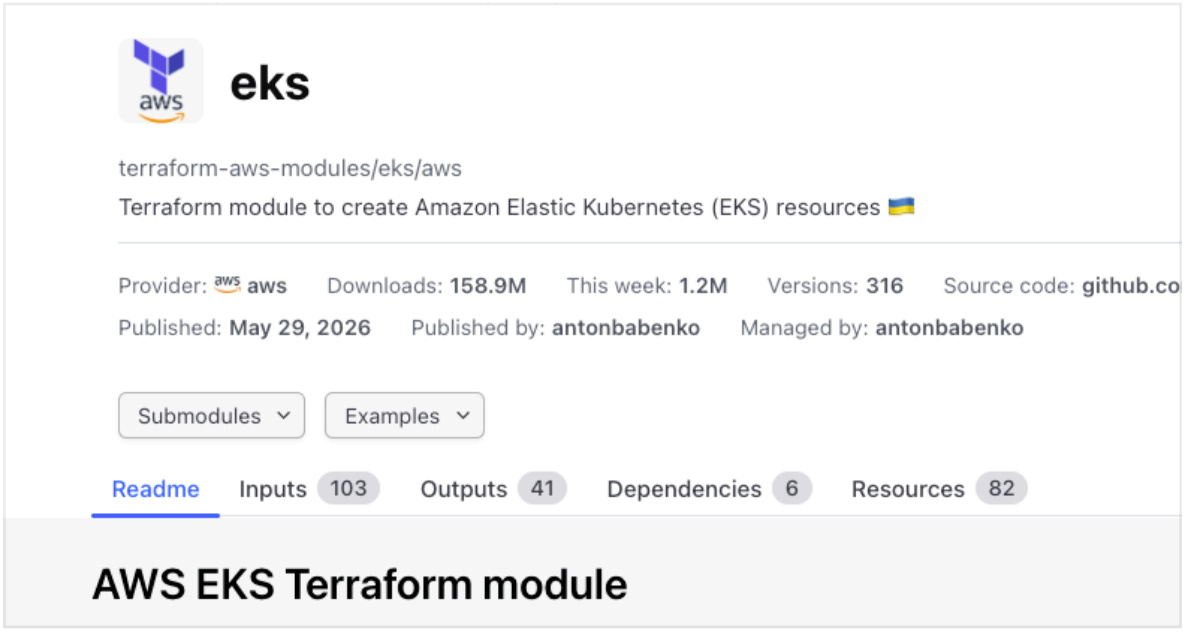
We will use the node groups managed by EKS with 'eks_manage_node_groups' attribute provided by the eks tf module.



And the description says "Map of EKS managed node group definitions to create"



And how do we know how to configure this object? -> we look at the examples of the documentation in the 'Readme' section



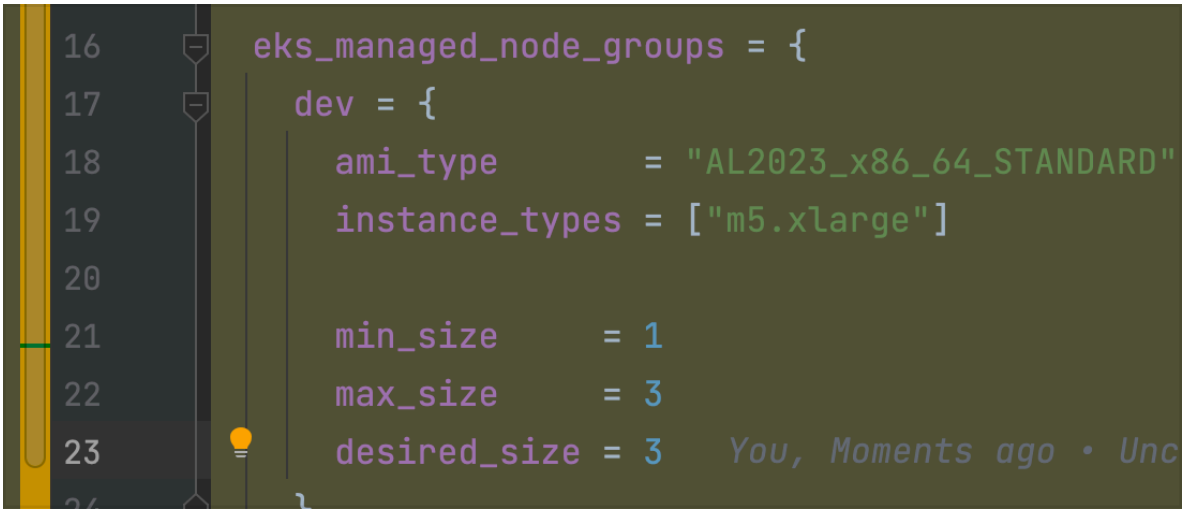
And we look for ‘eks_managed_node_groups’ example

```
  # EKS Managed Node Group(s)
  eks_managed_node_groups = {
    example = {
      # Starting on 1.30, AL2023 is the default AMI type for EKS managed node groups
      ami_type       = "AL2023_x86_64_STANDARD"
      instance_types = ["m5.xlarge"]

      min_size     = 2
      max_size     = 10
      desired_size = 2
    }
  }

  tags = {
    Environment = "dev"
    Terraform   = "true"
  }
}
```

We can have multiple node groups inside this attribute for example we can have a blue and a green deployment or we can have development, QA, etc -> and each group can have their own name and inside each group we can have one or more EC2 instances and we can define the configuration for this e instances.

So I copy the example and paste it and rename the group to ‘dev’

And we want to have minimum 1 instance, max 3 instances and the dried size is 3.



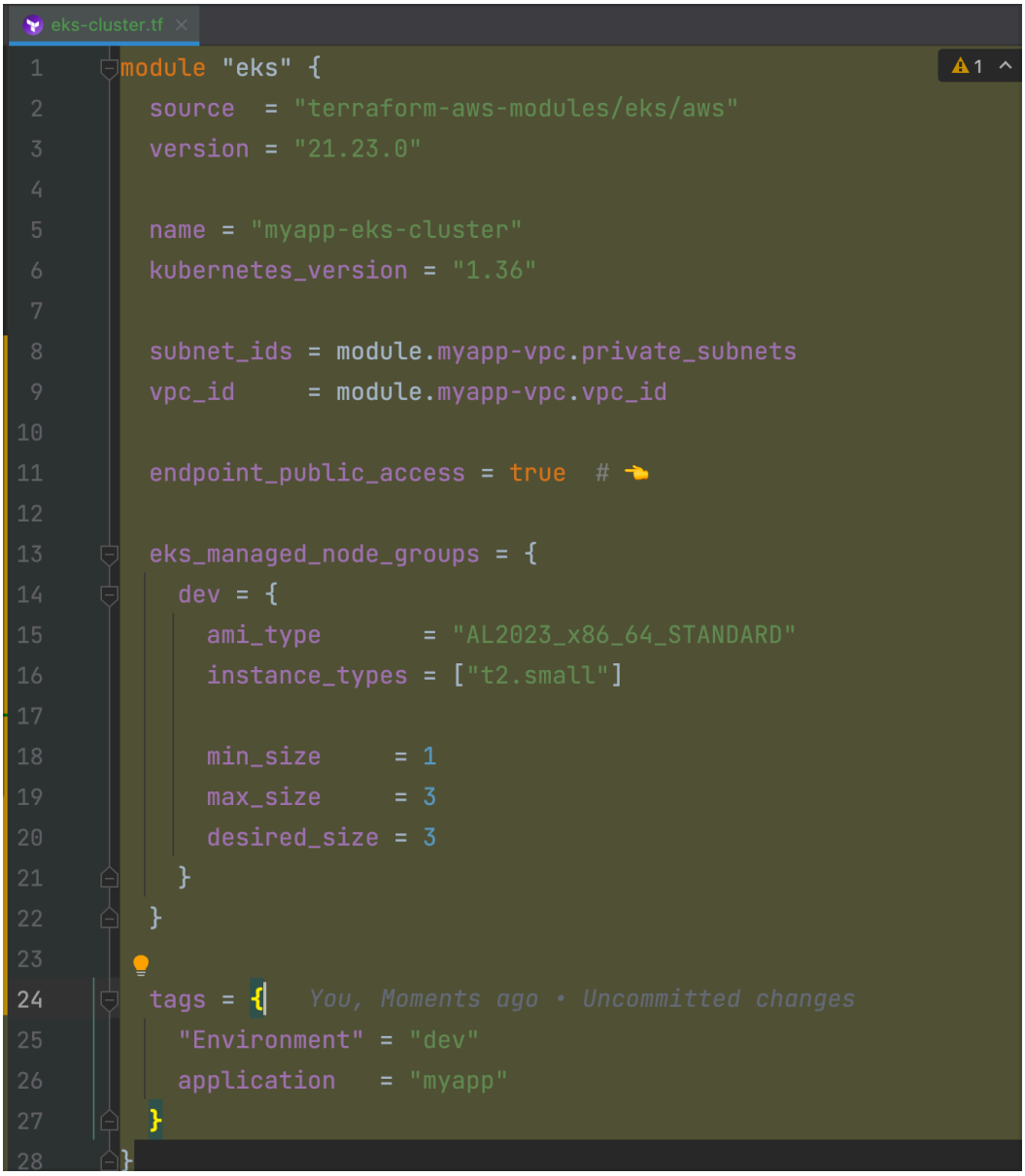
And we also will use t2.small instance type

```
16  eks_managed_node_groups = {
17    dev = {
18      ami_type       = "AL2023_x86_64_STANDARD"
19      instance_types = ["t2.small"]
20
21      min_size     = 1
22      max_size     = 3   You, A minute ago • Uncomm
```

We can enable some extra options on the cluster itself.

First we can set the cluster endpoint to be publicly accessible

-> endpoint_public_access = true -> means the Kubernetes API server (the endpoint) is reachable from the public internet.



We can also add
-> enable_cluster_creator_admin_permissions = true

```
  # Optional
  endpoint_public_access = true

  # Optional: Adds the current caller identity as an administrator via cluster access entry
  enable_cluster_creator_admin_permissions = true

  vpc_id                   = "vpc-1234556abcdef"
  subnet_ids               = ["subnet-abcde012", "subnet-bcde012a", "subnet-fghi345a"]
  control_plane_subnet_ids = ["subnet-xyzde987", "subnet-slkjf456", "subnet-qeiru789"]

  # EKS Managed Node Group(s)
  eks_managed_node_groups = {
    example = {
      # Starting on 1.30, AL2023 is the default AMI type for EKS managed node groups
      ami_type       = "AL2023_x86_64_STANDARD"
      instance_types = ["m5.xlarge"]

      min_size     = 2
      max_size     = 10
```

So whoever creates the cluster will have the permissions to manage the cluster automatically.

And once we create the cluster we will need to configure some workloads or install tools, so having the permissions will be needed.

— tangent —

This setting automatically grants the AWS identity that runs terraform apply full admin access to the EKS cluster.
-> enable_cluster_creator_admin_permissions = true

What it does?

When you create an EKS cluster, AWS only grants access to the IAM role/user that created it — but only via the older aws-auth ConfigMap mechanism. Without this, you might create the cluster and then find kubectl commands are denied.

With enable_cluster_creator_admin_permissions = true, the module:
1. Detects who is running Terraform (via aws sts get-caller-identity)
2. Creates a Cluster Access Entry granting that identity AmazonEKSClusterAdminPolicy

Access Entry vs aws-auth

EKS has two authentication approaches:

	aws-auth ConfigMap (old)	Access Entries (new)
Managed via	Kubernetes ConfigMap	AWS API
Introduced	EKS v1.0	EKS v1.29+
This setting uses	—	✅

When you'd set it to false

- Your CI/CD role already has access via a separate access entry
- You want to manage access entirely outside of Terraform
- You're using a shared Terraform role and don't want it to have cluster admin

Practical implication

If you set it to false (or omit it), after terraform apply you may not be able to run kubectl get nodes — you'd need to separately configure access for your IAM identity.

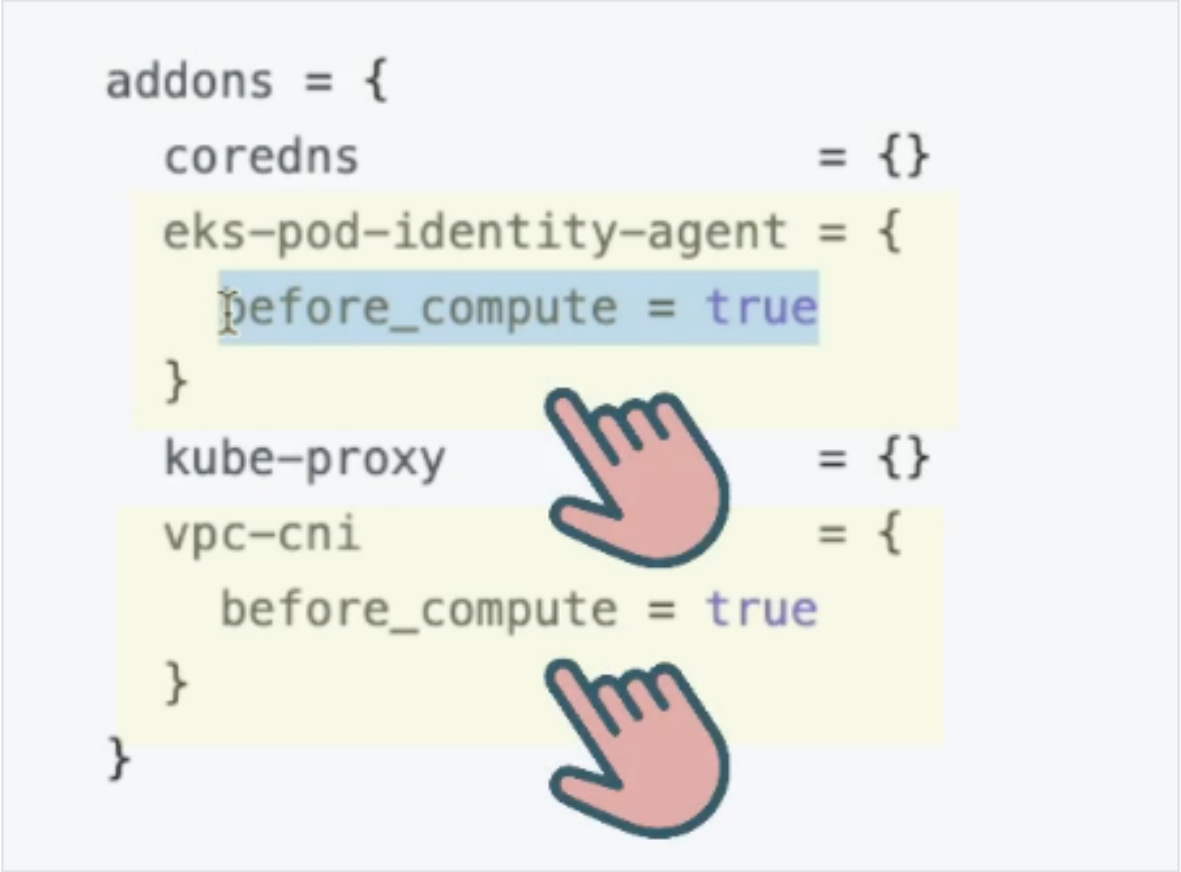
For most setups, leaving it true is the right default.

— end of tangent —

Finally we can define a set of addons that will be installed in our cluster. These are the core components that EKS relies on:

- Cordons
- Kube-proxy
- Vpc-cni
- Pod identity agent -> allows pods to use IAM roles from AWS without storing credentials.

So any workloads that we run her can access AWS services without us having to manage credentials.



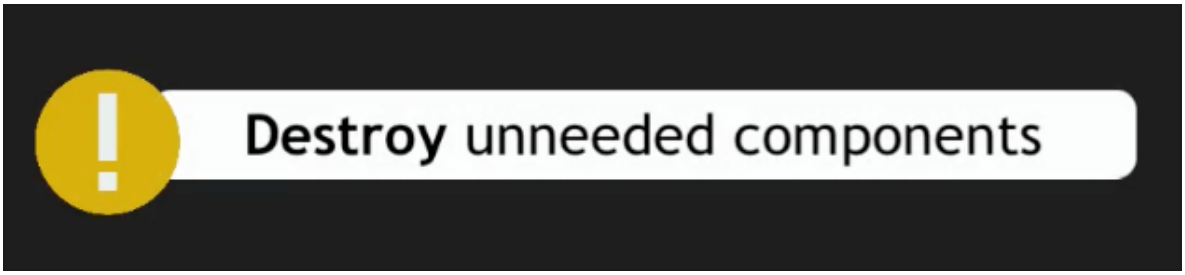
-> before-compute = true -> it means they get created before the worker nodes

```
  endpoint_public_access = true
  enable_cluster_creator_admin_permissions = true

  addons = {
    coredns                = {}
    eks-pod-identity-agent = {
      before_compute = true
    }
    kube-proxy             = {}
    vpc-cni                = {
      before_compute = true
    }
  }   You, 18 minutes ago • Uncommitted changes

  eks_managed_node_groups = {
    dev = {
      ami_type       = "AL2023_x86_64_STANDARD"
      instance_types = ["t2.small"]

      min_size     = 1
      max_size     = 3
      desired_size = 3
    }
```

MIND THE PRICING

EKS cluster + EC2instances = $ 💰

Make sure to destroy unneeded components



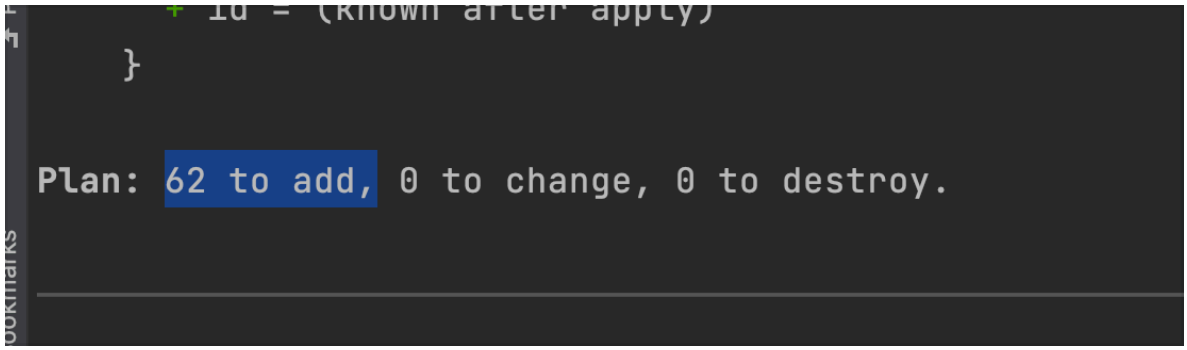
Apply configuration files

We added a new modele (eks module)
-> terraform init

```
Initializing modules...
Downloading registry.terraform.io/terraform-aws-modules/eks/aws 21.23.0 for eks...
- eks in .terraform/modules/eks
- eks.fargate_profile in .terraform/modules/eks/modules/fargate-profile
Downloading registry.terraform.io/terraform-aws-modules/kms/aws 4.0.0 for eks.kms...
- eks.kms in .terraform/modules/eks.kms
- eks.eks_managed_node_group in .terraform/modules/eks/modules/eks-managed-node-group
- eks.self_managed_node_group in .terraform/modules/eks/modules/self-managed-node-group
- eks.eks_managed_node_group.user_data in .terraform/modules/eks/modules/_user_data
- eks.self_managed_node_group.user_data in .terraform/modules/eks/modules/_user_data
Initializing provider plugins found in the configuration...
- Finding hashicorp/tls versions matching ">= 4.0.0"...
- Finding hashicorp/time versions matching ">= 0.9.0"...
- Finding hashicorp/cloudinit versions matching ">= 2.0.0"...
- Finding hashicorp/null versions matching ">= 3.0.0"...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Installing hashicorp/null v3.3.0...
- Installed hashicorp/null v3.3.0 (signed by HashiCorp)
- Using previously-installed hashicorp/aws v6.50.0
- Installing hashicorp/tls v4.3.0...
- Installed hashicorp/tls v4.3.0 (signed by HashiCorp)
- Installing hashicorp/time v0.14.0...
```

```
- Installing hashicorp/time v0.14.0...
- Installed hashicorp/time v0.14.0 (signed by HashiCorp)
- Installing hashicorp/cloudinit v2.4.0...
- Installed hashicorp/cloudinit v2.4.0 (signed by HashiCorp)

Initializing the backend...

Initializing provider plugins found in the state...

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

-> terraform plan



We don’t have to worry about defining all those resources that need to be created (62!) we just defined two modules and passed the parameters we were interested in.

This way we have a high level configuration that we can tweak with parameters.

When we created the cluster and the node group manually we had to create the role for them, TF via their modules takes care of defining the roles and how they should be created, so we don’t have to configure them.

```
  # module.eks.module.kms.data.aws_iam_policy_document.this[0] will be read during apply
  # (config refers to values not yet known)
<= data "aws_iam_policy_document" "this" {
      + id                        = (known after apply)
      + json                      = (known after apply)
      + minified_json             = (known after apply)
      + override_policy_documents = []
      + source_policy_documents   = []

      + statement {
          + actions   = [
              + "kms:*",
            ]
          + resources = [
              + "*",
```

Let’s apply the configuration
-> terraform apply

And it takes around ~15 minutes for the resources to be created

```
module.eks.aws_eks_addon.this["coredns"]: Creating...
module.eks.aws_eks_addon.this["kube-proxy"]: Creating...
module.eks.aws_eks_addon.this["coredns"]: Still creating... [00m10s elapsed]
module.eks.aws_eks_addon.this["kube-proxy"]: Still creating... [00m10s elapsed]
module.eks.aws_eks_addon.this["kube-proxy"]: Still creating... [00m20s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Still creating... [00m20s elapsed]
module.eks.aws_eks_addon.this["kube-proxy"]: Creation complete after 24s [id=myapp-eks-cluster:kube-proxy]
module.eks.aws_eks_addon.this["coredns"]: Still creating... [00m30s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Still creating... [00m40s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Still creating... [00m50s elapsed]
module.eks.aws_eks_addon.this["coredns"]: Creation complete after 55s [id=myapp-eks-cluster:coredns]

Apply complete! Resources: 62 added, 0 changed, 0 destroyed.
```